

1. The Agency Model

A generic AI assistant is a blank slate. It has no declared priorities, no persistent personality, no way to accumulate craft. Every session starts from zero. The agency system exists to change this. It gives agents *composable identities*—a role that defines what the agent does, paired with a motivation that defines why it acts the way it does. The same role combined with a different motivation produces a different agent. This is the key insight: identity is not a name tag, it is a *function*—the Cartesian product of competence and intent.

The result is an identity space that grows combinatorially. Four roles and four motivations yield sixteen distinct agents, each with its own behavioral signature. These identities are not administrative labels. They are content-hashed, immutable, evaluable, and evolvable. An agent's identity is a mathematical fact, verifiable by anyone who knows the hash.

1.1. Roles

A role answers a single question: *what does this agent do?*

It carries three identity-defining fields:

- **Description.** A prose statement of the role's purpose—what kind of work it performs, what domain it operates in, what skills it brings to bear.
- **Skills.** A list of skill references that define the role's capabilities. These are resolved at dispatch time and injected into the agent's prompt as concrete instructions (see Section 1.5 below).
- **Desired outcome.** What good output looks like. This is the standard against which the agent's work will be evaluated—not a vague aspiration, but a crisp definition of success.

A role also carries a *name* (a human-readable label like “Programmer” or “Architect”), a *performance* record (aggregated evaluation scores), and *lineage* metadata (evolutionary history). These are mutable—they can change without altering the role's identity. The name is for humans. The identity is for the system.

Consider two roles: one describes a code reviewer who checks for correctness, testing gaps, and style violations; the other describes an architect who evaluates structural decisions and dependency management. They may share some skills, but their descriptions and desired outcomes differ, so they produce different content-hash IDs—different identities, different agents, different behaviors when paired with the same motivation.

1.2. Motivations

A motivation answers the complementary question: *why does this agent act the way it does?*

Where a role defines competence, a motivation defines character. It carries three identity-defining fields:

- **Description.** What this motivation prioritizes—the values and principles that guide the agent's approach to work.
- **Acceptable trade-offs.** Compromises the agent may make. A “Fast” motivation might accept less thorough documentation. A “Careful” motivation might accept slower delivery. These are the negotiable costs of the agent's operating philosophy.
- **Unacceptable trade-offs.** Hard constraints the agent must never violate. A “Careful” motivation might refuse to ship untested code under any circumstances. A “Thorough” motivation might refuse to skip edge cases. These are non-negotiable.

Like roles, motivations carry a mutable name, performance record, and lineage. And like roles, only the identity-defining fields contribute to the content-hash.

The distinction between acceptable and unacceptable trade-offs is not decorative. When an agent's identity is rendered into a prompt, the acceptable trade-offs appear as *operational parameters*—flexibility the agent may exercise—and the unacceptable trade-offs appear as *non-negotiable constraints*—lines it must not cross. The motivation shapes behavior through the prompt: same role, different motivation, different output.

1.3. Agents: The Pairing

An agent is the unified identity in the agency system. For AI agents, it is the named pairing of exactly one role and exactly one motivation:

$$\text{agent} = \text{role} \times \text{motivation}$$

The agent's content-hash ID is computed from (`role_id`, `motivation_id`). Nothing else enters the hash. This means the agent is entirely determined by its constituents: if you know the role and the motivation, you know the agent.

An agent also carries operational fields that do not affect its identity:

Capabilities Flat string tags (e.g., "rust", "testing") used for task-to-agent matching at dispatch time. Capabilities are distinct from role skills: capabilities are for *routing* (which agent gets which task), skills are for *prompt injection* (what instructions the agent receives). An agent might have capabilities broader than its role's skills, or narrower, depending on how the operator configures it.

Rate An hourly rate for cost forecasting.

Capacity Maximum concurrent tasks this agent can handle.

Trust level A classification that affects dispatch priority (see Section 1.6 below).

Contact Email, Matrix ID, or other contact information—primarily for human agents.

Executor The backend that runs the agent's work (see Section 1.7 below).

The compositional nature of agents is what makes the identity space powerful. A "Programmer" role paired with a "Careful" motivation produces an agent that writes methodical, well-tested code and refuses to ship without tests. The same "Programmer" role paired with a "Fast" motivation produces an agent that prioritizes rapid delivery and accepts less thorough documentation. Both are programmers. They differ in *why* they program the way they do.

This is not a theoretical nicety. When the coordinator dispatches a task, the agent's full identity—role description, skills, desired outcome, acceptable trade-offs, non-negotiable constraints—is rendered into the prompt. The AI receives a complete behavioral specification before it sees the task. The motivation is not a hint; it is a contract.

1.4. Content-Hash IDs

Every role, motivation, and agent is identified by a SHA-256 hash of its identity-defining fields. The hash is computed from canonical YAML serialization of those fields, ensuring determinism across platforms and implementations.

Entity	Hashed fields
Role	description + skills + desired outcome
Motivation	description + acceptable trade-offs + unacceptable trade-offs
Agent	role ID + motivation ID

Table 1: Identity-defining fields for content-hash computation.

Three properties follow from content-hashing:

Deterministic. The same content always produces the same ID. If two people independently create a role with identical description, skills, and desired outcome, they get the same hash. There is no ambiguity, no namespace collision, no registration authority.

Deduplicating. You cannot create two entities with identical identity-defining fields. The system detects the collision and rejects the duplicate. This is not a bug—it is a feature. If two roles are identical in what they do, they *are* the same role. The name might differ, but the identity does not.

Immutable. Changing any identity-defining field produces a *new* entity with a new hash. The old entity remains untouched. This means you never “edit” an identity—you create a successor. The original is preserved, its performance history intact, its lineage available for inspection. Mutable fields (name, performance, lineage) can change freely without affecting the hash.

For display, IDs are shown as 8-character hexadecimal prefixes (e.g., a3f7c21d). All commands accept unique prefixes—you type as few characters as needed to disambiguate.

Why does this matter? Content-hashing makes identity a verifiable fact. You can confirm that two agents share the same role by comparing eight characters. You can trace an agent’s lineage through a chain of hashes, each linking to its parent. You can deduplicate across teams: if your colleague created the same role, the system knows. And because identity is immutable, performance data attached to a hash is *permanently* associated with a specific behavioral definition. A role’s score of 0.85 means something precise—it is the score of *that exact* description, *those exact* skills, *that exact* desired outcome.

1.5. The Skill System

Skills are capability references attached to a role. They serve double duty: they declare what the role can do (for humans reading the role definition), and they inject concrete instructions into the agent’s prompt (for the AI receiving the dispatch).

Four reference types exist:

Name A bare string label. "rust", "testing", "architecture". No content beyond the tag itself. Used when the skill is self-explanatory and needs no elaboration—the word *is* the instruction.

File A path to a document on disk. The file is read at dispatch time and its full content is injected into the prompt. Supports absolute paths, relative paths (resolved from the project root), and tilde expansion. Use this for project-specific style guides, coding standards, or domain knowledge that lives alongside the codebase.

Url An HTTP address. The content is fetched at dispatch time. Use this for shared resources that multiple projects reference—team-wide checklists, organization standards, living documents.

Inline Content embedded directly in the skill definition. The text is injected verbatim into the prompt. Use this for short, self-contained instructions: "Write in a clear, technical style" or "Always include error handling for network calls".

Skill resolution happens at dispatch time. Name skills pass through as labels. File skills read from disk. Url skills fetch over HTTP. Inline skills use their embedded text. If a resolution fails—a file is missing, a URL is unreachable—the system logs a warning but does not block execution. The agent is spawned with whatever skills resolved successfully.

The distinction between role skills and agent capabilities is worth emphasizing. *Skills* are prompt content—they are instructions injected into the AI's context. *Capabilities* are routing tags—they are flat strings compared against a task's required skills to determine which agent is a good match. An agent's capabilities might list "rust" and "testing" for routing purposes, while its role's skills include a detailed Rust style guide (as a File reference) and a testing checklist (as Inline content). The routing system sees tags; the agent sees documents.

1.6. Trust Levels

Every agent carries a trust level: one of **Verified**, **Provisional**, or **Unknown**.

Verified Fully trusted. The agent has a track record or has been explicitly vouched for. Verified agents receive a small scoring bonus in task-to-agent matching, making them more likely to be dispatched for contested work.

Provisional The default for newly created agents. Neither trusted nor distrusted. Most agents start here and stay here unless explicitly promoted.

Unknown External or unverified. An agent from outside the team, or one that has not yet demonstrated reliability. Unknown agents receive no penalty—they simply lack the bonus that Verified agents enjoy.

Trust is set at agent creation time and can be changed later. It does not affect the agent's content-hash ID—trust is an operational classification, not an identity property.

1.7. Human and AI Agents

The agency system does not distinguish between human and AI agents at the identity level. Both are entries in the same agent registry. Both can have roles, motivations, capabilities, and trust levels. Both are tracked, evaluated, and appear in the synergy matrix. The identity model is uniform.

The difference is the **executor**—the backend that delivers work to the agent.

claude The default. Pipes a rendered prompt into the Claude CLI. The agent is an AI. Its role and motivation are injected into the prompt, shaping behavior through language.

matrix Sends a notification via the Matrix protocol. The agent is a human who monitors a Matrix room.

email Sends a notification via email. The agent is a human who checks their inbox.

shell Runs a shell command from the task's exec field. The agent is a human (or a script) that responds to a trigger.

For AI agents, role and motivation are *required*—an AI without identity is a blank slate, which is precisely what the agency system exists to prevent. For human agents, role and motivation are *optional*. Humans bring their own judgment, priorities, and character. A human agent might have a role (to signal what kind of work to route to them) or might operate without one (receiving any work that matches their capabilities).

Both types are evaluated using the same rubric. But human agent evaluations are excluded from the evolution signal—the system does not attempt to “improve” humans through the evolutionary process. Evolution operates only on AI identities, where changing the role or motivation has a direct, mechanistic effect on behavior through prompt injection.

1.8. Composition in Practice

To make the compositional nature of agents concrete, consider a small agency seeded with `wg agency init`. This creates four starter roles and four starter motivations:

Starter Roles	Starter Motivations
Programmer	Careful
Reviewer	Fast
Documenter	Thorough
Architect	Balanced

Table 2: The sixteen possible pairings from four roles and four motivations.

A “Programmer” paired with “Careful” produces an agent that writes methodical, tested code and treats untested output as a hard constraint violation. The same “Programmer” paired with “Fast” produces an agent that ships quickly and accepts less documentation as a reasonable trade-off. A “Reviewer” with “Thorough” examines every edge case and refuses to approve incomplete coverage. A “Reviewer” with “Balanced” weighs thoroughness against schedule pressure and accepts pragmatic compromises.

Each of these sixteen pairings has a unique content-hash ID. Each accumulates its own performance history. Over time, the evaluation data reveals which combinations excel at which kinds of work—the synergy matrix (detailed in *Section 5*) makes this visible. High-performing pairs are dispatched more often. Low-performing pairs are candidates for evolution or retirement.

The same compositionality applies to evolved entities. When the evolver mutates a role—say, refining the “Programmer” description to emphasize error handling—a *new* role is created with a new hash. Every agent that referenced the old role continues to exist unchanged. New agents can be created pairing the refined role with existing motivations. The old and new coexist, each with their own performance records, until the evidence shows which is superior.

1.9. Lineage and Deduplication

Content-hash IDs enable two properties that matter at scale: lineage tracking and deduplication.

Lineage. Every role, motivation, and agent records its evolutionary ancestry. A manually created entity has no parents and generation zero. A mutated entity records one parent and increments the generation. A crossover entity records two parents and increments from the highest. The `created_by` field distinguishes human creation (“human”) from evolutionary creation (“evolver-`{run_id}`”).

Because identity is content-hashed, lineage is unfalsifiable. The parent entity cannot be silently altered—any change would produce a different hash, breaking the lineage link. You can walk the ancestry chain from any entity back to its manually created roots, confident that each link refers to the exact content that existed at creation time. This is not a version history in the traditional sense. It is an immutable record of how the agency’s identity space has evolved.

Deduplication. If the evolver proposes a role that is identical to an existing one—same description, same skills, same desired outcome—the content-hash collision is detected and the duplicate is rejected. This prevents the agency from accumulating redundant entities. It also means that convergent evolution is recognized: if two independent mutation paths arrive at the same role definition, the system knows they are the same role.

1.10. Federation: Sharing Across Projects

An agency built in one project is not confined to that project. The federation system lets you share roles, motivations, and agents across workgraph projects—transferring proven identities from one context into another, complete with their performance histories and lineage chains.

Federation operates through named *remotes*: references to another project’s agency store, managed via `wg agency remote add`, `wg agency remote list`, and `wg agency remote remove`. Remotes are stored in `.workgraph/federation.yaml`. Once a remote is configured, three operations become available.

Scanning. `wg agency scan <remote>` lists the roles, motivations, and agents in a remote store without modifying anything. This is reconnaissance—you see what exists before deciding what to import.

Pulling. `wg agency pull <remote>` copies entities from the remote store into the local project. Roles, motivations, agents, and their evaluation records are all transferred. You can filter by entity type (`--roles-only`, `--motivations-only`) or by specific entity IDs. A `--dry-run` flag previews the operation without writing.

Pushing. `wg agency push <remote>` is the symmetric operation—it copies local entities to the remote store. The same filtering and dry-run options apply.

Content-hash IDs make federation natural. Because identity is determined by content, the same role has the same ID in every project. When you pull a role that already exists locally, the system recognizes the collision and skips the duplicate. There is no mapping table, no namespace negotiation, no manual reconciliation. Identity deduplication is a mathematical consequence of content-hashing.

The interesting question is what happens to *metadata*—the mutable fields that sit outside the content-hash. Performance records are merged: evaluation references from both stores are unioned, deduplicated by the `(task_id, timestamp)` tuple, and average scores are recalculated from the merged set. This means pulling from a remote enriches the local performance picture—you gain evaluation data from contexts you have never seen. Lineage is preserved by preferring the richer ancestry: if the remote’s lineage records more parents or a higher generation, it takes precedence. Names default to keeping the local value, though a `--force` flag overrides this.

Referential integrity is enforced during transfer. When you pull an agent, its referenced role and motivation are automatically included—you cannot end up with an agent pointing to a role that does not exist. If a dependency is missing from the source store, the operation fails with a clear error rather than creating a broken reference.

Federation preserves lineage across project boundaries. An entity pulled from a remote carries its full ancestry chain. You can trace it back through mutations and crossovers to its manually created roots, even when those roots were created in a different project by a different team. The immutable nature of content-hash IDs guarantees that each link in the chain refers to the exact content that existed at creation time, no matter where it was created.

The practical effect is that organizations can maintain a shared pool of proven agent identities. A team that has evolved an effective “Reviewer” role over dozens of evaluations can push it to a shared remote. Other teams pull it, pair it with their own motivations, and immediately benefit from that evolutionary history. The performance data travels with the entity, so the receiving team can see *why* the role is considered effective before deciding to adopt it.

1.11. Cross-References

The agency model described here is the *identity layer* of the system. How these identities are dispatched to tasks—the claim-before-spawn protocol, the wrapper script, the coordinator’s tick loop—is detailed in *Section 4*. How agents are evaluated after completing work, and how evaluation data feeds back into evolution, is detailed in *Section 5*.

One detail bridges the agency model and the evaluation system: every evaluation carries a source field that identifies where the score came from. Internal auto-evaluations have source “`llm`”. External signals use structured tags—“`outcome:sharpe`” for market data, “`ci:test-suite`” for CI results, “`vx:peer-id`” for peer assessments. The source field is a freeform string, not a closed enum, so any signal source can participate. This matters for the agency model because an agent’s performance record aggregates evaluations from *all* sources. The evolver sees the full picture: internal quality assessments alongside external outcome data. The interplay between diverse evaluation sources and the evolutionary process is detailed in *Section 5*.